

고성모

게임 · 임베디드 · 로봇틱스를 넘나드는 소프트웨어 개발자

복잡한 시스템을 구조로 나누고, 하드웨어부터 게임 로직까지 직접 동작시키는 것에 집중합니다.

Unity 게임 아키텍처를 중심으로, ESP32 임베디드 펌웨어와 ROS2 로봇틱스까지 여러 도메인의 프로젝트를 완주했습니다.

Unity · C# 게임 개발

ESP-IDF · C 임베디드

ROS2 로봇틱스

Android · Java

대표 역량 게임 클라이언트 아키텍처 · 임베디드 하드웨어 브링업

주요 언어 C# · C / C++ · Java · Python

대표 프로젝트 괴이탐사국: INSPECTOR · 바른자세 코치봇 · ROS2 AGV

01 핵심 역량 & 프로젝트 한눈에

한 언어·한 분야에 머물지 않고, “무엇을 만들든 구조를 설계하고 끝까지 동작시킨다”는 방식으로 게임·임베디드·로보틱스·모바일을 관통하는 공통 역량을 쌓았습니다.

집중 스킬 매트릭스

구분	보유 스킬 (★ = 집중 영역)
언어	C# ★ · C / C++ ★(임베디드) · Java · Python
엔진 · 프레임워크	Unity ★ · ESP-IDF / FreeRTOS ★ · ROS2 · Android SDK
설계 · 패턴	상태 머신(FSM) · 싱글톤 / CRTP 제네릭 · 데이터 주도 설계 · partial class 모듈화 · 이벤트 기반 결합
데이터 · 저장	JSON · ScriptableObject · SQLite · ContentProvider · PlayerPrefs · CSV 로깅
하드웨어 · 통신	I2C(PCA9685 서보) · I2S(마이크 / 스피커) · GPIO / PWM · UART · 차동구동 기구학
도구 · 협업	Git / GitHub 브랜치 전략 · PR 리뷰 · 크로스플랫폼 빌드(macOS ↔ Windows) · QA / 디버깅

프로젝트 요약

프로젝트	유형	스택 · 핵심	역할
괴이탐사국: INSPECTOR	팀 · 캡스톤 게임	Unity·C# — 전투 엔진 / 데이터 주도 아키텍처	클라이언트 개발 (단독)
바른자세 코치봇	임베디드 AI 로봇	ESP32-S3·ESP-IDF·C — 서보 보행 / AI 음성	펌웨어·브링업
ROS2 운반 로봇(AGV)	팀 · 로보틱스	ROS2 — 라인트레이싱 / 차동구동 설계	문제정의·설계
FPS 프로토타입	학습 게임	Unity·C# — FSM 적 AI / NavMesh	구현
Android 앱 3종	학습 모바일	Java — SQLite · ContentProvider · Canvas	구현

깊이 — 게임 아키텍처

약 19,500줄 규모 전투 시스템을 partial class·제네릭·데이터 주도로 설계. 대규모 코드를 나누는 힘.

폭 — 하드웨어 브링업

I2C 서보, I2S 오디오, 4족 보행을 베어 메탈 C로 직접 구동. AI 프레임워크 크로스빌드까지.

기초 — CS 펀더멘털

DB·ContentProvider·그래픽·로봇 기구학 등 분야를 가리지 않는 기본기.

02 대표 프로젝트 ① — 괴이탐사국: INSPECTOR

로그라이크 덩크빌딩 오토배틀러

Unity · C# · 팀 578(3인) 캡스톤 · 클라이언트 개발 단독 담당 · 2026.03-06

카드는 **전투 변수(스택)** 만 만들고 동료가 **자동으로** 싸우는 “간접 전투” 게임. 기획 명세를 실제 동작하는 전투 엔진·시스템 코드로 단독 구현했습니다.

100

C# 스크립트

19.5K

코드 라인

78

개인 커밋

13+

기능 브랜치·PR

설계 하이라이트

① 전투 엔진을 partial class로 분리

약 2,665줄로 커진 `BattleManager` 를 5개 파일로 책임 분리 — 흐름(`.Phases`)과 계산(`.Combat`), 적 AI(`.EnemyAction`)를 격리해 유지보수성 확보.

② 반복 패턴을 CRTP 제네릭으로 일반화

`Singleton<T>` · `BaseCurrency<T>` · `RoleCostBase<T>` 로 인스턴스 관리·저장·UI 갱신의 중복을 제거. 타입 안전 + 중복 인스턴스 자동 제거.

③ 데이터 주도 설계 (JSON + SO)

동료·스킬·적 정의를 JSON으로 빼내 기획자가 코드 없이 밸런싱. DB 5종이 `Dictionary` 캐싱·조회만 담당하고 상태는 SO가 관리.

④ 데이터로 조종하는 적 AI

가중치 랜덤 스킬 선택 + 타겟팅 매트릭스(도발·소환·독). `weight` · `targeting` 값만 바꾸면 AI 재구성. 4스킬 보스 구현.

전투 페이즈 상태 머신

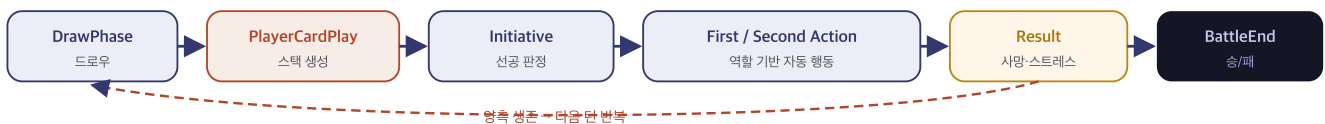


그림 1. `BattlePhase` enum 상태 머신 — 각 상태를 `.Phases.cs` 의 `Handle*` 핸들러로 대응

사용 스킬

C#

Unity

partial class

CRTP 제네릭

데이터 주도(JSON+SO)

상태 머신

이벤트 기반 결합

Git 브랜치·PR

QA 반복 대응

03 대표 프로젝트 ② — 바른자세 코치봇

ESP32-S3 기반 AI 4족 자세 코치 로봇

ESP-IDF v5.5.4 · C / C++ · FreeRTOS · Xiaozhi AI · 펌웨어 & 하드웨어 브링업

센서가 **앞으로 숙인 자세** 를 감지하면 **AI 음성** 으로 자세를 교정해 주는 4족 로봇. 보드 점등부터 서보 보행·오디오·AI 연동까지 하드웨어를 바닥부터 직접 올렸습니다.

단계별 하드웨어 브링업 파이프라인

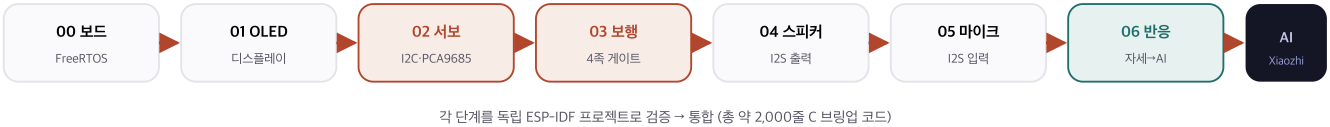


그림 2. 위험을 한 단계씩 줄이는 브링업 전략 — 하나가 확실히 동작한 뒤 다음 하드웨어로 진행

베어메탈 I2C 서보 제어

라이브러리 없이 `driver/i2c` 로 `PCA9685` 16채널 PWM 컨트롤러(0x40)를 직접 제어해 다리 서보 각도를 명령. 이를 시퀀스로 엮어 **4족 보행 게이트** 구현(`03_walk_test` , 463줄).

I2S 오디오 입출력

`i2s_std` 로 마이크 캡처·스피커 재생을 구성하고, 자세 감지 이벤트에 **음성 반응**을 연결(`06_mic_reaction_test` , 477줄).

AI 프레임워크 이식 · 크로스빌드

Xiaozhi AI 펌웨어(디바이스 상태머신·`mcp_server` ·OTA)를 **Windows 전용 가이드**를 macOS로 이식해 ESP-IDF v5.5.4로 빌드 성공.

재현 가능한 실행 환경 구성

`demo.sh` · `flash_robot.sh` 로 포트 자동 탐색·플래시·모니터를 스크립트화 — **함정**(읽기전용 폴더·`IDF_TARGET`·포트)을 문서화해 재현성 확보.

사용 스킬

C / C++

ESP-IDF

FreeRTOS

I2C / PCA9685

I2S 오디오

GPIO·PWM 서보

크로스플랫폼 빌드

Bash 자동화

AI 음성 연동

04 프로젝트 ③④⑤ — 로보틱스 · 게임 · 모바일

대표 프로젝트 외에도 로봇 시스템 설계, 게임 AI, 모바일 기초를 각각의 프로젝트로 다뤄 폭넓은 CS 기본기를 검증했습니다.

③ ROS2 운반 로봇(AGV) 설계 & 로보틱스 기구학

자율주행 운반 로봇 프로토타입 정의

공정 부품 운송 자동화를 위한 라인트레이싱 AGV 문제정의·요구사항 설계. 경로 인식, CSV 주행 데이터 로깅, 초음파/LiDAR 안전정지, MES-PLC 연동 규격, 1시간 구동 등 실무형 요구 도출.

ROS2

AGV 시스템 설계

차동구동 기구학

센서·제어 이론

로봇 기구학·제어 이론

차동구동($v = \omega R$), 라디안, 엔코더, 모터, 센서, 마이크로컨트롤러, 제어 등 9개 주제를 출처 교차검증하며 정리. ROS2 참고 코드 포함.

④ FPS 프로토타입 — 게임 AI & 전투

FSM 기반 적 AI

EnemyFSM 으로 대기→추격→공격→피격→복귀 상태를 전이하고, NavMesh 경로탐색·Animator 연동으로 자연스러운 추적 구현.

C#

Unity

FSM AI

NavMesh

Animator

플레이어·전투 시스템

이동·회전·발사(PlayerFire)·카메라 추적, 스폰너·폭발·피격 이벤트 등 FPS 코어 루프 전반을 12개 스크립트로 구성.

⑤ Android 앱 3종 — 데이터 & 그래픽 기초

앱	핵심 기술	구현
SQLite DB 앱	로컬 데이터베이스	DBHelper 로 테이블 생성·CRUD
ContentProvider 앱	앱 간 데이터 공유	PersonProvider + DBHelper 연동
그래픽 앱	커스텀 뷰 · Canvas	ImageDisplayView 로 직접 드로잉

Java

Android SDK

SQLite

ContentProvider

Canvas 그래픽

05 집중 스킬 심층 정리

여러 프로젝트를 관통하는 세 가지 집중 스킬과, 그것을 뒷받침하는 실제 근거를 정리했습니다.

① 대규모 코드를 “나누는” 아키텍처 감각 — 가장 깊은 역량

- **모듈 분리**: 2,665줄 전투 시스템을 partial class 5개로, 비대한 데이터 클래스를 `.Hp/.Skill/.Stress` 로 분할.
- **중복 일반화**: 싱글톤·재화·스택의 공통 로직을 **CRTP 제네릭 베이스**로 추출.
- **관심사 분리**: 정의 데이터(JSON)와 상태(SO), 로직과 표현(이벤트 기반)을 분리.

근거

INSPECTOR — `BattleManager.*`, `Singleton<T>`, `BaseCurrency<T>`, 5종 JSON 데이터베이스

② 하드웨어를 바닥부터 올리는 임베디드 브리징 — 두 번째 집중 영역

- **단계적 검증**: 보드→OLED→서보→보행→오디오→AI를 독립 프로젝트로 하나씩 확인 후 통합.
- **저수준 통신**: 라이브러리 없이 I2C(PCA9685)·I2S를 레지스터 수준에서 제어.
- **환경 정복**: Windows 전용 빌드를 macOS로 이식하고, 실패 함정을 스크립트·문서로 재현 가능하게 정리.

근거

바른자세 코치봇 — `02_servo_test` (312줄)·`03_walk_test` (463줄)·`06_mic_reaction_test` (477줄), ESP-IDF v5.5.4 이식

③ 분야를 가리지 않는 CS 기본기 & 협업 — 넓은 기반

- **상태 머신**을 게임 전투(BattlePhase)와 게임 AI(EnemyFSM), 디바이스 제어에 반복 적용.
- **데이터 계층**을 JSON·SQLite·ContentProvider·CSV로 상황에 맞게 선택.
- **협업**: Git 기능별 브랜치·PR, QA 피드백 15+항목 반복 반영, 팀 문서를 코드로 번역.

근거

FPS(FSM·NavMesh) · Android(SQLite·Provider·Canvas) · ROS2(기구학·설계) · INSPECTOR(팀 협업)

06 개발 방식 — AI 협업 개발(바이브 코딩)

이 프로젝트들의 상당 부분은 AI 협업 개발(바이브 코딩)으로 구현했습니다. 도구가 코드를 빠르게 써 주는 만큼, 제 역할은 **무엇을 왜 만들지 정하고, 결과를 검증·통합·디버깅** 하는 데 있었습니다. 이 점을 숨기지 않고, 하나의 **실전 역량**이자 학습의 출발점으로 정리합니다.

역할 분담 — 내가 한 것 vs AI가 가속한 것

작업	주도	내용
요구 정의 · 문제 분해	본인	무엇을·왜 만들지, 완료 기준 설정
아키텍처 · 구조 설계 판단	본인	partial 분리·데이터 주도·단계별 브리핑 등 결정
반복 구현 · 보일러플레이트	AI 가속	정의된 구조를 코드로 빠르게 채움
환경 설정 · 빌드 함정 해결	협업	내 검증 + AI 제안 (ESP-IDF·MCP 등)
코드 리뷰 · 통합 · 디버깅	본인	동작 검증, 버그 추적, 조각 통합
에디터 · 하드웨어 조작	MCP 자동화	Unity MCP·플래시 스크립트를 내 지시로 실행

사용한 AI 협업 도구

Claude Code · Unity MCP(에디터 자동 조작) · ESP-IDF 워크플로우 · `execute_code` (애니메이터·NavMesh 빌드) · 플래시/데모 Bash 자동화.

나의 바이브 코딩 루프

① MCP로 현재 상태 확인 → ② 작업 이해를 내 말로 정리 → ③ 확인·승인 → ④ 실행 후 직접 검증·수정. 이해하지 못한 채 복붙하지 않는 것을 원칙으로 합니다.

이 방식을 어떻게 바라보는가 (정직하게)

AI 협업 개발은 도구 오케스트레이션 · 프롬프트 설계 · MCP 자동화라는 실전 역량이라고 생각합니다. 동시에, 이렇게 빠르게 “만들 수 있게” 된 것과 “밑바닥까지 이해한 것” 사이엔 아직 간극이 있음을 인정합니다. 그 간극을 메우기 위한 학습 계획을 다음 장에 정리했습니다.

07 학습 가이드 & 성장 계획

목표는 분명합니다 — “AI와 함께 만들 수 있다”를 “도구 없이 설명하고 스스로 다시 만들 수 있다”로 바꾸는 것. 바이브 코딩으로 다룬 개념을 도메인별로 다시 학습할 계획입니다.

도메인별 학습 계획

영역	지금 (바이브로 구현)	학습 목표	방법 · 자료
C# / OOP	CRTP·제네릭·이벤트 사용	제네릭·델리게이트 원리 이해	MS Docs · 『C# in Depth』 · 재구현
자료구조·알고리즘	List·Dictionary 활용	시간복잡도·자료구조 직접 구현	백준·알고리즘 문제 · 노트 정리
디자인 패턴	싱글톤·상태머신 적용	패턴 의도·트레이드오프 이해	『Head First Design Patterns』
Unity 내부	MonoBehaviour·SO 사용	생명주기·코루틴·메모리·프로파일링	Unity Learn · 프로파일러 실습
임베디드 C	I2C/I2S 브링업	포인터·메모리·RTOS 스케줄링 원리	ESP-IDF 문서 · 데이터시트 직독
로봇 제어	차동구동 조사	기구학·PID 제어 직접 구현	ROS2 튜토리얼 · 제어이론 강의

학습 원칙 — 바이브 코딩을 이해로 전환하는 3단계

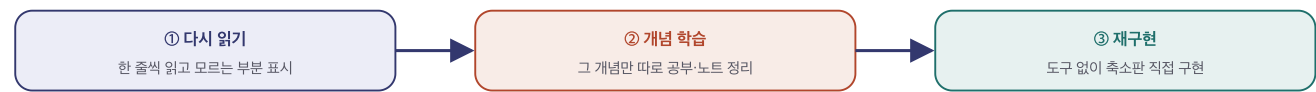


그림 3. “읽고 → 이해하고 → 스스로 다시 만든다” — 이해 없이 넘어간 코드를 남기지 않는 학습 루프

지향점
AI 협업으로 빠르게 완주하는 실행력과, 그 위에 스스로 설명·재구현하는 기본기를 함께 갖춰 — “하드웨어를 이해하고, AI를 도구로 부리는 소프트웨어 개발자”로 성장하려 합니다.